

Merging DataFrames, Data Mining, Scrappy & Beautiful Soup



by
Dr M Rajasekhara Babu

School of
Computer Science and Engineering
<https://www.rajababu.org>

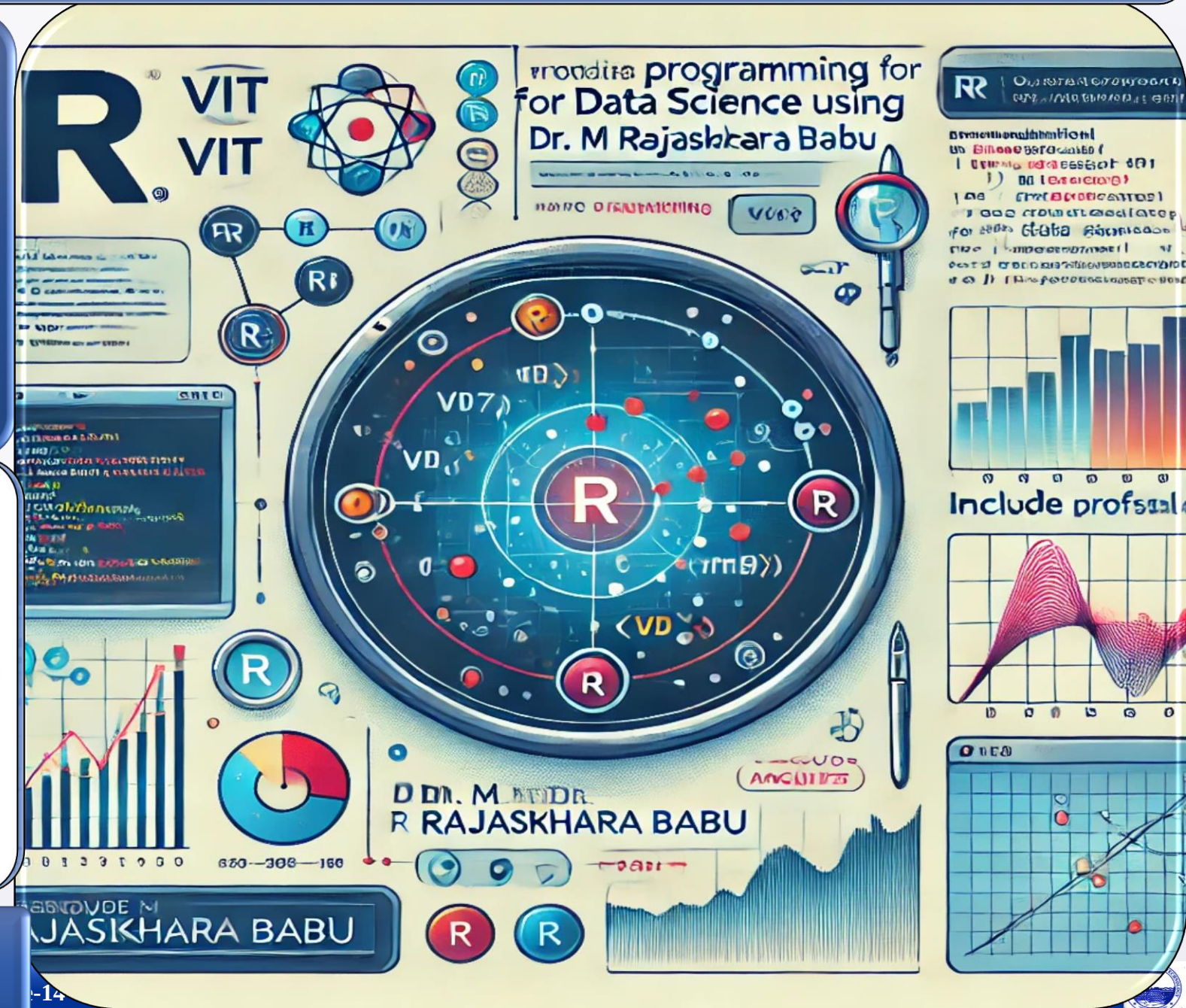


VIT[®]

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

Vellore-632014, Tamil Nadu, India



Why Merge Data?

Real-world data rarely lives in one place. Combining datasets unlocks richer analysis and deeper insights.

Student Info Table

ID, Name — identity records stored separately from performance data.

Student Marks Table

ID, Marks — academic results linked by a shared key.

After Merge

ID, Name, Marks — a unified view enabling complete analysis.

Creating Sample DataFrames

- Before merging, we create two simple DataFrames using `pd.DataFrame()`.
- Each shares an ID column that will serve as the merge key.

```
import pandas as pd
```

```
students = pd.DataFrame({  
    'ID': [1, 2, 3],  
    'Name': ['John', 'Mary', 'David']  
})
```

```
marks = pd.DataFrame({  
    'ID': [1, 2, 3],  
    'Marks': [85, 90, 78]  
})
```

```
print(students)  
print(marks)
```

Merging DataFrames with `merge()`

The `merge()` Function

Use `pd.merge()` to combine two DataFrames on a shared key column.

```
result = pd.merge(  
    students,  
    marks,  
    on='ID'  
)  
print(result)
```

**Students
dataframe**

ID	Name
1	John
2	Mary
3	David

**Marks
dataframe**

ID	Name
1	85
2	90
3	78

Output

ID	Name	Marks
1	John	85
2	Mary	90
3	David	78

The result is a single unified DataFrame combining both sources.

Types of Joins

The `how` parameter in `pd.merge()` controls which records are included in the result.

1

Inner Join

Returns matching records only

```
how='inner'
```

2

Left Join

Returns all records from the left

```
how='left'
```

3

Right Join

Returns all records from the right

```
how='right'
```

4

Outer Join

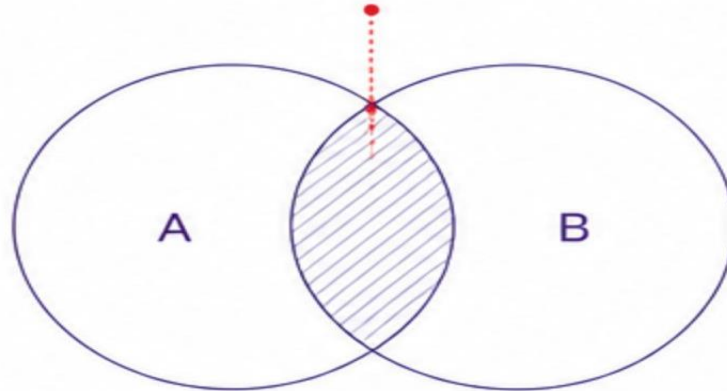
Returns all records from both

```
how='outer'
```

Join Visualization & Use Cases

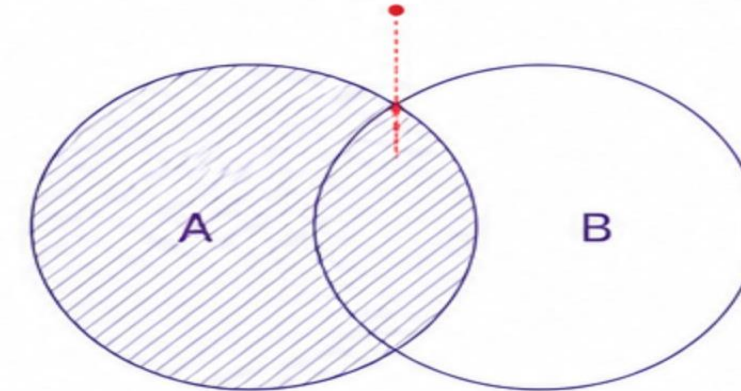
Inner Join

(Table A intersection Table B - matching records only)



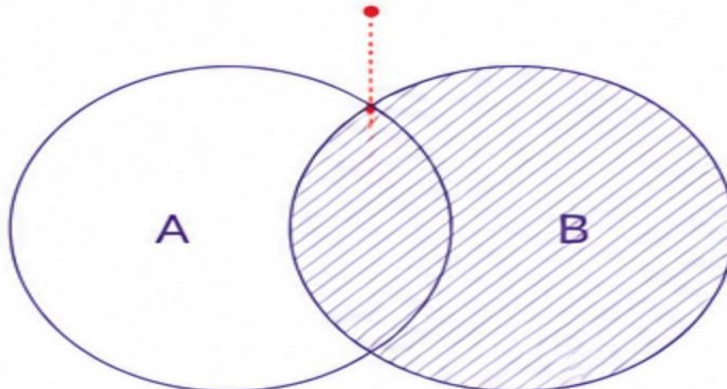
Left Join

(All of A plus matching B)



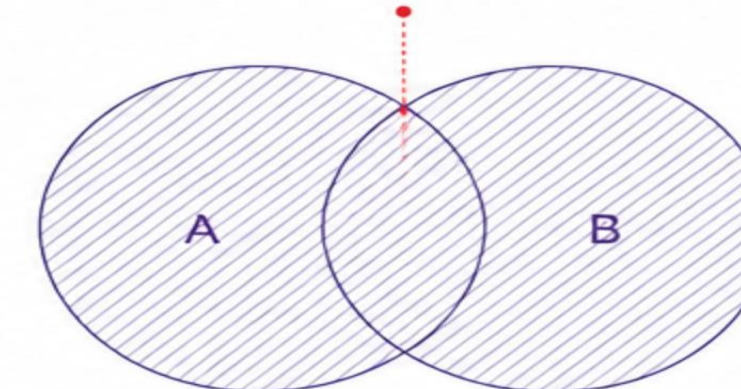
Right Join

(All of B plus matching A)



Outer Join

(All records from both A and B)



Student Databases

Healthcare Data

E-Commerce Transactions

Concatenating DataFrames

The `concat()` Function

Combines DataFrames by stacking rows or columns — no shared key required.

```
# Row-wise (stack vertically)  
pd.concat([df1, df2])
```

```
# Column-wise (stack horizontally)  
pd.concat([df1, df2], axis=1)
```

Merge vs. Concat

Merge	Concat
Uses common key	No key required
Relational operation	Stacking operation
Similar to SQL JOIN	Similar to combining files
Complex analysis	Data appending

Applications: monthly sales reports, multi-semester student records.

Introduction to Data Mining

- **Data Mining** is the process of discovering useful patterns, trends, and relationships from large datasets — transforming raw data into valuable information.
- **Data Collection**
Gather raw data from various sources.
- **Data Cleaning & Integration**
Remove noise and combine datasets.
- **Pattern Discovery**
Apply algorithms to uncover hidden insights.
- **Decision Making**
Use discovered patterns to drive action.



Applications of Data Mining



Education

- Student performance prediction
- Dropout analysis



Banking

- Fraud detection
- Credit scoring



Healthcare

- Disease prediction
- Patient monitoring

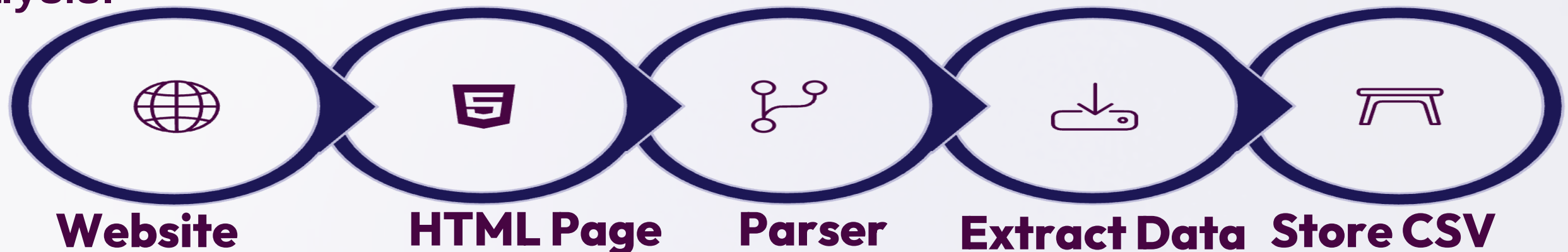


E-Commerce

- Product recommendation
- Customer segmentation

Introduction to Web Scraping

Web scraping is the automatic extraction of information from websites. Websites contain a wealth of structured data — news articles, product information, weather reports, and job postings — that can be extracted for analysis.



Understanding HTML Structure

Web scrapers must understand HTML to locate and extract the right data. Key tags include headings, paragraphs, links, and lists.

Example HTML

```
<h1>Welcome</h1>
<p>Data Science Course</p>
<a href="url">Visit</a>
<ul>
  <li>Python</li>
  <li>Data Science</li>
</ul>
```

Important Tags

Tag	Purpose
<h1>–<h6>	Headings
<p>	Paragraphs
<a>	Hyperlinks
/	Lists
<div>	Containers

Introduction to Beautiful Soup

What is Beautiful Soup?

Beautiful Soup is a Python library for parsing HTML and XML, searching webpage elements, and extracting

```
pip install beautifulsoup4
```

Basic Usage

```
from bs4 import BeautifulSoup

html = "<h1>Welcome</h1>"
soup = BeautifulSoup(
    html, "html.parser"
)
print(soup.h1.text)
# Output: Welcome
```

Key Methods

```
find()
```

Locate the first matching tag:

```
soup.find("h1")
```

```
find_all()
```

Locate all matching tags:

```
soup.find_all("p")
```

```
.text
```

Extract the text content from any tag.

Extracting Links with BeautifulSoup

Beautiful Soup makes it simple to find and extract hyperlinks from any HTML page using the `find()` method and accessing tag attributes.

HTML Source

```
<a href="https://example.com">  
  Visit Python  
</a>
```

Python Code

```
link = soup.find("a")  
print(link['href'])
```

Output

```
https://example.com
```

Extracting a List — Full Example

```
from bs4 import BeautifulSoup
```

```
html = """  
<h2>Courses</h2>  
<ul>  
  <li>Python</li>  
  <li>Data Science</li>  
  <li>AI</li>  
</ul>"""
```

```
soup = BeautifulSoup(  
    html, 'html.parser')
```

```
for item in soup.find_all("li"):  
    print(item.text)
```

```
# Output: Python, Data Science, AI
```


Introduction to Scrapy

Scrapy is an open-source Python framework designed for large-scale web scraping, data extraction, and crawling multiple web pages efficiently.



Fast

Asynchronous architecture for high-speed crawling.



Extensible

Middleware and pipelines for custom workflows.



Robust

Built-in error handling and retry mechanisms.



Industrial-Grade

Suitable for production-scale applications.

Scrapy Architecture & Setup

Architecture Flow



- A Spider is the core class that defines how to crawl a site and extract data from its pages.

Getting Started

```
# Install Scrapy  
pip install scrapy
```

```
# Create a new project  
scrapy startproject demo
```

```
# Generate a spider  
scrapy genspider sample  
example.com
```

Writing a Simple Scrapy Spider

A Scrapy spider defines the starting URLs and a `parse()` method that processes each downloaded page and yields extracted data.

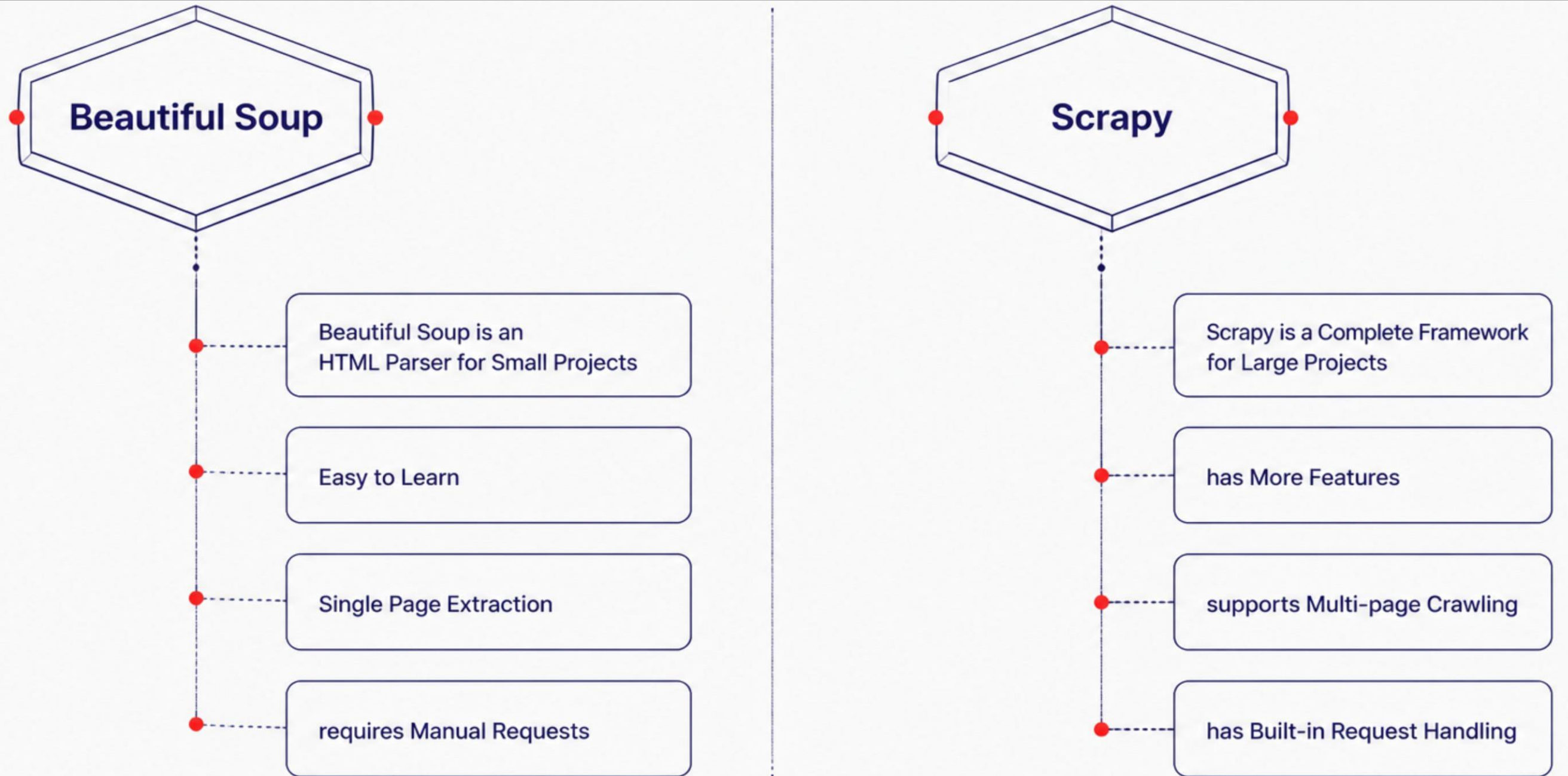
```
import scrapy

class DemoSpider(scrapy.Spider):
    name = "demo"
    start_urls = [
        "https://example.com"
    ]

    def parse(self, response):
        title = response.css(
            "h1::text"
        ).get()
        yield {
            "Title": title
        }
```

The `response.css()` selector uses CSS syntax to target elements, and `yield` outputs each extracted record as a dictionary.

Beautiful Soup vs. Scrapy



Choose BeautifulSoup for quick, small-scale parsing tasks. Choose Scrapy when you need to crawl multiple pages at scale with built-in request handling and data pipelines.

The Task

A website contains structured course data: Course Name, Instructor Name, and Duration.

- Extract course details using BeautifulSoup
- Store results in a Pandas DataFrame
- Analyze the collected course data

Tools Used

Beautiful Soup

Parse and extract HTML content from the course page.

Pandas

Store extracted data in a structured DataFrame for analysis.

Activity 1 — DataFrame Merging

Create two DataFrames: Student Details (ID, Name) and Student Marks (ID, Marks). Merge them using:

- Inner Join
- Left Join

Activity 2 — HTML Parsing

Parse the following HTML using

```
<h1>Data Science</h1>  
<p>Learning Python</p>
```

Extract the heading text and the paragraph text using `find()`.

Real-World Applications

The skills from this module power solutions across industries — from education analytics to competitive intelligence.



Education

Course catalog analysis and student performance analytics.



Business

Competitor price monitoring and market intelligence.



Research

Publication and academic data collection at scale.



Recruitment

Job portal analytics and talent market insights.

